



# COMP 432: Introduction to Machine Learning

## Team 7: Randos

## Project Report

### **Prepared By**

Syed Ayaan Jilani - 40209519

Luqman Hakim - 40196877

Romain Garnier – 40349310

COMP 432 – DD

Fall 2025

Professor Ali Ayub, PhD.

Due Date – 28<sup>th</sup> November 2025

Concordia University

Montreal, QC, Canada

## Table of Contents

|      |                                                            |    |
|------|------------------------------------------------------------|----|
| I.   | Introduction .....                                         | 1  |
| II.  | Chronological stages in the model's creation .....         | 1  |
| III. | Data Pre-processing.....                                   | 2  |
|      | Initial Exploration.....                                   | 2  |
|      | Feature Selection .....                                    | 3  |
|      | Principal Component Analysis.....                          | 4  |
| IV.  | Model architecture and Training .....                      | 5  |
|      | Decision Tree Baseline.....                                | 5  |
|      | MLP Architecture .....                                     | 6  |
|      | Training Methodology.....                                  | 7  |
|      | Loss Function .....                                        | 7  |
|      | Optimization .....                                         | 7  |
|      | Regularization .....                                       | 8  |
|      | Hyperparameter Tuning .....                                | 8  |
| V.   | Evaluation.....                                            | 8  |
|      | Metrics .....                                              | 8  |
|      | Comparative Insights.....                                  | 8  |
|      | Error Analysis .....                                       | 10 |
|      | Strong Classes .....                                       | 10 |
|      | Depth and Size Effects .....                               | 12 |
| VI.  | Conclusion.....                                            | 13 |
| VII. | Appendix.....                                              | 14 |
|      | Precision and recall on the last version of the code ..... | 14 |

## List of Figures

|                                                                                          |    |
|------------------------------------------------------------------------------------------|----|
| Figure 1: Header of the data set .....                                                   | 2  |
| Figure 2 : information about the shape and features of dataset using panda library ..... | 3  |
| Figure 3: Summary statistics of the dataset features. ....                               | 3  |
| Figure 4: Correlation matrix of the dataset .....                                        | 4  |
| Figure 5: Cumulative variance explained by principal components.....                     | 5  |
| Figure 5: vertical 'boîte à moustache' of the feature 100 for the class [46,21,29] ..... | 11 |
| Figure 6 : vertical 'boîte à moustache' of the feature 78 for the class [46,21,29] ..... | 11 |
| Figure 7: confusion matrix of the validation set of the best model.....                  | 12 |

## List of Tables

|                                                                                    |   |
|------------------------------------------------------------------------------------|---|
| Table 1: Performance metrics of the scratch decision tree on the training set..... | 6 |
| Table 2: Class wise performance on the training set. ....                          | 9 |
| Table 3: Class wise performance on the validation set for the best model. ....     | 9 |

## I. Introduction

In this project, a supervised classification problem from the COMP 432 Kaggle competition was studied. The dataset provided to us contains numerical feature vectors that were extracted from deep vision models and associated with an object class among fifty possible categories. Each sample is described by five hundred real valued features, which makes the task a high dimensional classification problem. The objective that was set by the course was to design and implement models from scratch, using tools such as pytorch and numerical libraries, and to obtain the highest possible accuracy on the hidden Kaggle test set, while respecting the constraints on model design and academic integrity. Accuracy on the private leaderboard is used to determine part of the project grade, which motivates careful attention to generalization rather than only to performance on the training data.

To address this task, a sequence of stages was followed. The dataset was first explored using the pandas library in order to inspect basic statistics, shapes and potential correlations between features and labels. Simple feature selection based on correlation was attempted, followed by principal component analysis, in order to evaluate whether dimensionality reduction could simplify the problem without discarding useful information. A decision tree classifier implemented from scratch was then used as a baseline, which revealed the difficulty of the fifty-class problem. After this initial exploration, the focus was shifted to a multi layer perceptron architecture, where the depth, width and regularization mechanisms were tuned. The final part of the work concentrated on systematic evaluation through k fold cross validation, error analysis by class, and the study of depth and size effects on the performance of the model.

## II. Chronological stages in the model's creation

Here is a timeline of the important steps in the project's development. The architecture details, precise explanations and choices motivations will be developed in the next parts of this report.

- First, one proceeded to **data analysis**, using here panda's library, to display the features of the data set and main information.
- Having taken the course on decision trees and seeing that the dataset contained 500 features, a first idea that came in mind was initially to **try implementing a decision tree by filtering the features and keeping only the most correlated with the label** to be predicted (50 possible classes). For this, we relied on a code snippet originating from the lab 5 of COMP432 Machine Learning course, which relies on using a correlation matrix to select features and also a code snippet from lab 6 about Classification Decision trees from scratch. The results were not very

encouraging, as we obtained an accuracy score of only 7% on the training set. Rather than continuing with this approach, we decided to switch to a neural network.

- **We then defined a Multi-Layer Perceptron (MLP) neural network** comprising a shallow network of three fully connected layers. The score was immediately better: 47% on the public leaderboard.
- To increase the neural network's performance, **we kept expanding the network by adding more layers**, but we quickly encountered **overfitting**.
- To avoid overfitting, we chose to add **regularization** to the model, and we settled with **AdamW** optimizer. The resulting score was 61% on the public leaderboard.
- The initial idea of filtering features then came back to mind, with the aim of potentially increasing the score if certain features were to be avoided. By representing the code that relies on the correlation matrix used for the decision tree, **we added a selection of the 300 features most correlated with the label. The results were less favourable**, at around 47%.
- To better understand which features to keep, we performed a **PCA analysis** on our training set. We concluded that we would need to keep approximately 450 features to explain 95% of the variance, which led us to reject the idea of filtering features: it seems difficult to remove features without losing information. This confirms and complements our observations from studying the correlation matrix: the first feature was at -18%, the top 10 most correlated features were above 10%, and then the correlation smoothly dropped: none clearly stood out as sufficiently significant, and we couldn't remove many because those we kept wouldn't be any more significant for the classification task.
- We then **kept the model with 61% accuracy score**, since it meets the requirements of this project and shows an accuracy score for the test data close to the validation accuracy score.

### III. Data Pre-processing

#### Initial Exploration

The dataset was first explored using the Pandas library to understand its structure, feature distributions, and potential correlations with the target label. With 500 features, dimensionality reduction was considered early in the process.

|   | id | feature_0 | feature_1 | ... | feature_498 | feature_499 | label |
|---|----|-----------|-----------|-----|-------------|-------------|-------|
| 0 | 0  | 0.000000  | 0.000000  | ... | 0.0         | 0.063888    | 43    |
| 1 | 1  | 0.000000  | 0.000000  | ... | 0.0         | 0.000000    | 16    |
| 2 | 2  | 0.111111  | 0.000000  | ... | 0.0         | 0.000000    | 21    |
| 3 | 3  | 0.000000  | 0.087039  | ... | 0.0         | 0.000000    | 2     |
| 4 | 4  | 0.000000  | 0.000000  | ... | 0.0         | 0.000000    | 1     |

Figure 1: Header of the data set

```

Shape: (115406, 502)
Columns: Index(['id', 'feature_0', 'feature_1', 'feature_2', 'feature_3', 'feature_4',
               'feature_5', 'feature_6', 'feature_7', 'feature_8',
               ...,
               'feature_491', 'feature_492', 'feature_493', 'feature_494',
               'feature_495', 'feature_496', 'feature_497', 'feature_498',
               'feature_499', 'label'],
              dtype='object', length=502)

```

Figure 2 : information about the shape and features of dataset using panda library

|       | id           | feature_0     | ... | feature_499   | label         |
|-------|--------------|---------------|-----|---------------|---------------|
| count | 115406.00000 | 115406.000000 | ... | 115406.000000 | 115406.000000 |
| mean  | 57702.50000  | 0.019694      | ... | 0.009752      | 24.501265     |
| std   | 33314.98692  | 0.048032      | ... | 0.030195      | 14.430520     |
| min   | 0.00000      | 0.000000      | ... | 0.000000      | 0.000000      |
| 25%   | 28851.25000  | 0.000000      | ... | 0.000000      | 12.000000     |
| 50%   | 57702.50000  | 0.000000      | ... | 0.000000      | 25.000000     |
| 75%   | 86553.75000  | 0.000000      | ... | 0.000000      | 37.000000     |
| max   | 115405.00000 | 0.635001      | ... | 0.436436      | 49.000000     |

Figure 3: Summary statistics of the dataset features.

## Feature Selection

A correlation matrix was computed to identify the top 10 features most correlated with the label. However, the highest correlation observed was only -18%, with the top 10 features barely exceeding 10%. This suggested that no single feature was strongly predictive, complicating feature selection.

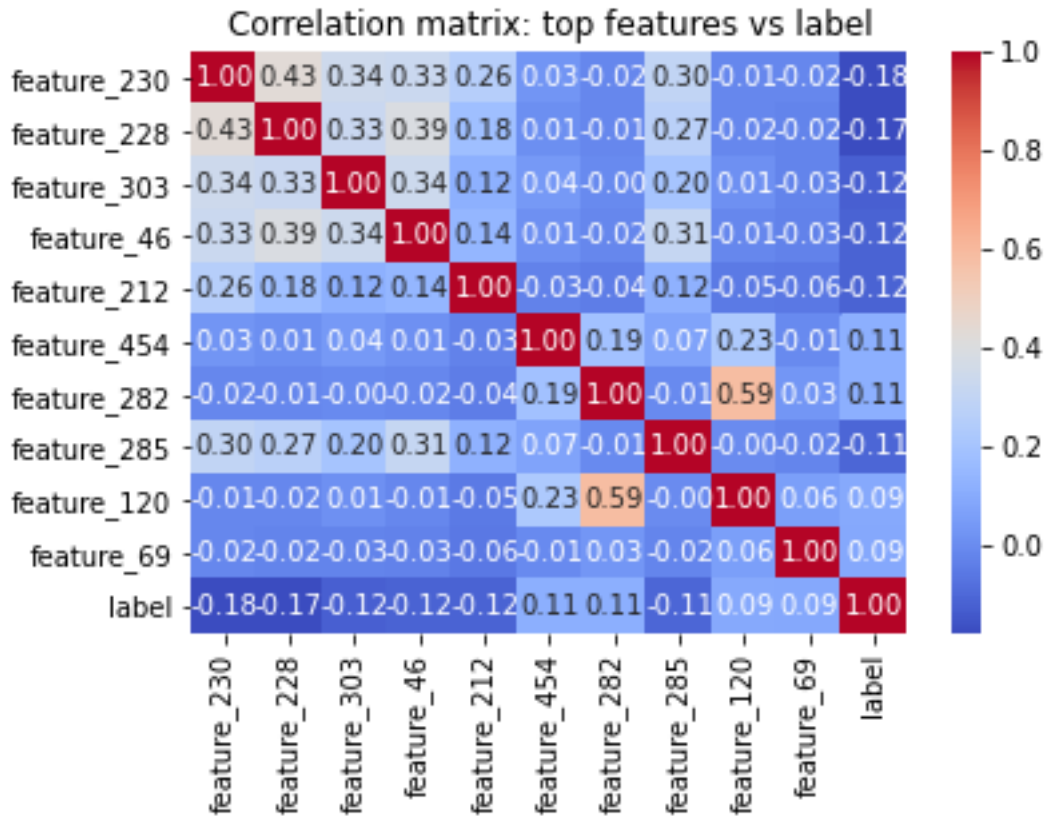


Figure 4: Correlation matrix of the dataset

## Principal Component Analysis

Principal component analysis (PCA) was used to examine whether the original feature space could be reduced while preserving most of the information in the data. PCA constructs new orthogonal directions, called principal components, that successively capture the largest possible amount of variance in the standardized features. In this way, PCA provides a quantitative view of how concentrated or spread out the information is across the different directions in feature space.

PCA was applied to the standardized training features, using the covariance matrix of the transformed data and its eigenvalues and eigenvectors. The eigenvalues were sorted in decreasing order, and the cumulative explained variance ratio was computed as more components were added. The resulting curve is shown in Figure 5. The cumulative explained variance increases relatively smoothly, and approximately ninety five percent of the total variance is only reached when about four hundred fifty principal components are retained. In other words, a large fraction of the components contributes non negligible variance, and there is no small subset of components that captures most of the information.

This behaviour suggests that the original features are not strongly redundant and that many of them carry distinct information. Therefore, aggressive dimensionality reduction would discard useful signal and is unlikely to help model performance. At the

same time, retaining four hundred fifty principal components would reduce the input dimension by only a modest amount compared with the original five hundred features, while also making the transformed features harder to interpret. For these reasons, PCA was treated primarily as an exploratory tool in this project, and the subsequent neural network models were trained on the full standardized feature set rather than on a PCA reduced representation.

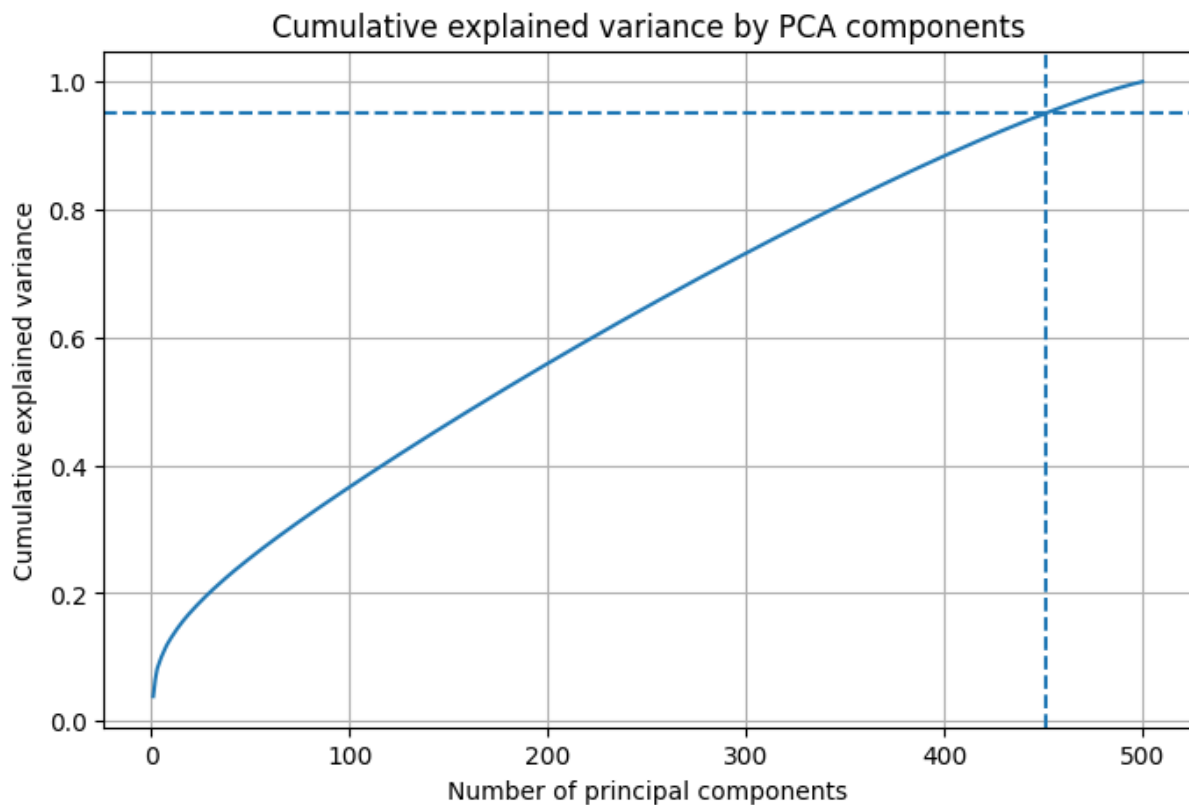


Figure 5: Cumulative variance explained by principal components.

## IV. Model architecture and Training

### Decision Tree Baseline

Initially, a decision tree classifier was implemented using the top 7 correlated features. However, the model achieved only 7% accuracy on the training set, likely due to the limited predictive power of the selected features and the complexity of the 50-class problem.

The Decision Tree used is based on the one built from scratch in lab 5. The decision tree architecture is based on a greedy algorithm that divides the dataset in two regions and call recursively itself on each of this region. At each call, one optimizes the split by minimizing the error - maximizing the gain - finding a threshold ( $s = \text{best\_criteria}[1]$ ) and a dimension (index  $j = \text{best\_criteria}[0]$ ) to look at. To avoid a too complex tree and avoid



overfitting, one considers only some features (**K=7**) given by the correlation analysis and set a **max depth (10)**. (the running time was still dozens of minutes)

Mains functions of the code are

- The fit function, defined line 159, for training the model on a dataset proceeds then:
    - First, test for the base cases, when the max depth is reached or if there is only one feature left, one just returns the majority class
    - Optimize the split by looping over the feature index and threshold and calculating the gain for each, keeping the higher one.
    - If there is no gain, one returns the majority class, otherwise one calls recursively the fit function on the left and right region of the split dataset and return a node with the left and right children and the criteria used for the split.
  - The Gini impurity function, defined line 119, and the entropy function, defined line 128, to compute the gain
  - The split\_dataset function defined line 137 separates the current dataset in two regions (left and right), given a threshold  $s$  and a dimension index `feature_index`
  - The predict function defined line 215 run through the decision tree for each sample of the considered dataset (train, validation or test set) to predict the corresponding class.
- ➔ The model was assessed using `accuracy_score` and `classification_report` function from `sklearn.metrics`.

Scratch Decision Tree Accuracy: 0.07445886695665736 then 7.4% on the train set !

Table 1: Performance metrics of the scratch decision tree on the training set.

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| macro avg    | 0.30      | 0.07   | 0.07     | 115406  |
| weighted avg | 0.30      | 0.07   | 0.07     | 115406  |

The low accuracy, low precision and low recall, with already at long time to run code, lead us to consider Neural Networks and more especially the Multi-Layer Perceptron.

## MLP Architecture

Since the dataset consisted of numerical feature vectors extracted from deep vision models, we chose Multi-Layer Perceptron as the most appropriate model since it is specifically designed for tabular, dense, and non-spatial feature data, making it well suited for our case.

We initially experimented with a shallow network of **three fully connected layers**. This was a good start as we got an accuracy score of 47% on the public leaderboard – very

close to the high baseline. Gradually, we explained our network to **five fully connected layers (500 -> 1024 -> 512 -> 256 -> 128 -> 50)** which gave us our best accuracy score of 61% on the public leaderboard.

To ensure that our MLP remained stable and prevented overfitting, each block has -

- **Batch Normalisation** – normalizes activation of layers by scaling the outputs based on batch's mean and variance, accelerating convergence
- **ReLU activation** – introduces non-linearity helping the network learn complex patterns
- **Dropout** – sets certain neuron activations to zero randomly, which prevents overfitting to training data by forcing the network to learn features on its own rather than relying on specific neurons

In each block, we implemented the following sequence: **Layer -> Batch Normalization -> ReLU -> Dropout**. This order ensures that the outputs from the layers are normalized and sent to ReLU. After activation, the outputs are effectively regularized by dropouts.

Overall, this architecture struck the right balance between efficiently utilizing the current dataset and generalization ability.

### Training Methodology

The model was trained for 20 epochs as the validation accuracy plateaued around this number. Increasing the epochs to 25-30 introduced mild overfitting and marginally less accuracy score on the validation set and public leaderboard. Stopping at 20 epochs ensured efficient computation while avoiding unnecessary training.

To ensure reliable performance and good generalization ability of the model, we introduced K-Fold cross-validation with K = 5 as using a single validation split could lead to high variance in the results. Cross-validation ensured robust performance of the model across multiple splits and confidence that the model generalizes well to unseen data. The best performing model after the cross-validation was used for final evaluation on the train set, validation set and the public leaderboard.

### Loss Function

We used **Cross-Entropy** loss, which is predominantly used for multi-class classification problems. It measures the difference between the predicted class probabilities and the true class probabilities. Cross-Entropy heavily penalizes predictions that were wrong and made with high confidence, helping prevent overly wrong predictions.

### Optimization

We tested three optimizers -

- **Adam** initially gave good performance but upon research, we learnt that Adam applies weight decay on top of the gradient, which can interfere with Adam's adaptive learning rates.

- **AdamW** provided us with the best performance. It decouples weight decay from gradient update, making regularization more effective resulting in better generalization and stability.
- **SGD** performed even lower than Adam during our experiments. While SGD can outperform Adam with carefully tuned hyperparameters, this would require extensive hyperparameter tuning and given the competition's time constraints, the benefit was not worth the cost.

## Regularization

As discussed in the earlier section, each MLP block has **Batch Normalization and Dropout** to stabilize training and reduce overfitting. Along with that, we apply **Weight Decay (L2 regularization)** through AdamW which penalizes large weights and helps the model generalize better on unseen data.

## Hyperparameter Tuning

We experimented with a wide range of learning rates and weight decay by implementing a simple manual grid search. The resulting values were **LR = 0.001** and **WD = 1e-4** as they resulted in smooth and stable convergence along with good generalization during K-Fold cross-validation.

# V. Evaluation

## Metrics

Accuracy (percentage of correct predictions) was the primary metric, given the balanced nature of the dataset. Precision and recall were also monitored to identify class-specific strengths and weaknesses.

## Comparative Insights

The MLP with AdamW and k fold cross validation achieved 61% accuracy, a significant improvement over the decision tree (7%).

To assess the predictions of the model, one can use the `accuracy_score` and `classification_report` functions from `sklearn.metrics`. The `classification_report` function computes the recall and the precision of the model for each class. They are defined as the following:

Precision: (also called positive predictive value) is the fraction of relevant instances among the retrieved instances. Written as a formula:

$$Precision = \frac{\text{Relevant retrieved instances}}{\text{All retrieved instances}}$$

Recall: (also known as sensitivity) is the fraction of relevant instances that were retrieved. Written as a formula:

$$Recall = \frac{Relevant\ retrieved\ instances}{All\ relevant\ instances}$$

(Source [https://en.wikipedia.org/wiki/Precision\\_and\\_recall](https://en.wikipedia.org/wiki/Precision_and_recall))

Because the last version is the one using k-folds, one decided to assess both the predictions of all the train set and the validation set of the best model kept among all the folds. Thus, one can compare the results and see if there is a potential overfit on the train data.

The complete results are in **Erreur ! Source du renvoi introuvable..** Here are the main observations and related conclusions:

Table 2: Class wise performance on the training set.

|      | precision | recall   | f1-score | support  |
|------|-----------|----------|----------|----------|
| mean | 0.9198    | 0.919    | 0.9202   | 2308.12  |
| std  | 0.024285  | 0.021595 | 0.021332 | 1.847668 |
| min  | 0.86      | 0.87     | 0.87     | 2303     |
| max  | 0.97      | 0.97     | 0.97     | 2310     |

Table 3: Class wise performance on the validation set for the best model.

|      | precision | recall   | f1-score | support  |
|------|-----------|----------|----------|----------|
| mean | 0.6148    | 0.6126   | 0.6122   | 461.62   |
| std  | 0.101945  | 0.097558 | 0.097673 | 21.80141 |
| min  | 0.41      | 0.42     | 0.43     | 400      |
| max  | 0.85      | 0.84     | 0.84     | 495      |

- The accuracy scores are similar for all the folds (61% with two significant figures); thus, the data set is quite homogeneous
- For the train set, one observes very high precision and high recall, 92% each. That means that the model hasn't any tendency to have high recall low precision or high precision and low recall. The small standard deviation provides that the model works the same on all classes.
- For the validation set of the best model, the precision and recall are still similar but quite lower than on the train set. The 30% difference means that our model overfit the train data. That could be one of the main areas of improvement.
- However, the accuracy is consistently same across the K-Fold cross-validation with very little difference with the accuracy on the public leaderboard (~61%).
- This indicates moderately good generalization on unseen data; one considers the model is consistent enough to be kept regarding the threshold of 51%.

## **Error Analysis**

For both assessments, the minimum value for the precision score (0.86 and 0.41) is reached for class 29. In the validation set, this 20% difference with the mean points out that this class is problematic for the model. That means the model often predicts class 29 when it is a different class (many false positive). A possible explanation could be feature overlap: class 29 would share similar characteristics with other classes → The model "confuses" these classes and predicts 29 as a precaution.

For both assessments, the minimum value for the recall score (0.87 and 0.42) is reached for class 21. In the validation set, this 19% difference with the mean points out that this class is problematic for the model. That means the model does not detect class 21 sufficiently (many false negative). Possible explanations could be 1) underrepresentation in the training set: Class 21 may have fewer samples, so the model hasn't learned it well. 2) Non-discriminating features: The features of class 21 are very similar to other classes, or too noisy. Looking at the repartition of the class, class 21 doesn't have less samples than the others (479, around the others), then 1) explanation is not verified.

## **Strong Classes**

For both assessments, the maximum precision and recall is reached for class 46. That means that the model has well learnt this class. The values on the validation set are still high, which means that this class is identified robustly even with new data.

One can compare the repartitions of a feature for those classes using a 'boîte à moustache'. It shows the median and quartiles and aberrant data. Here are the results:

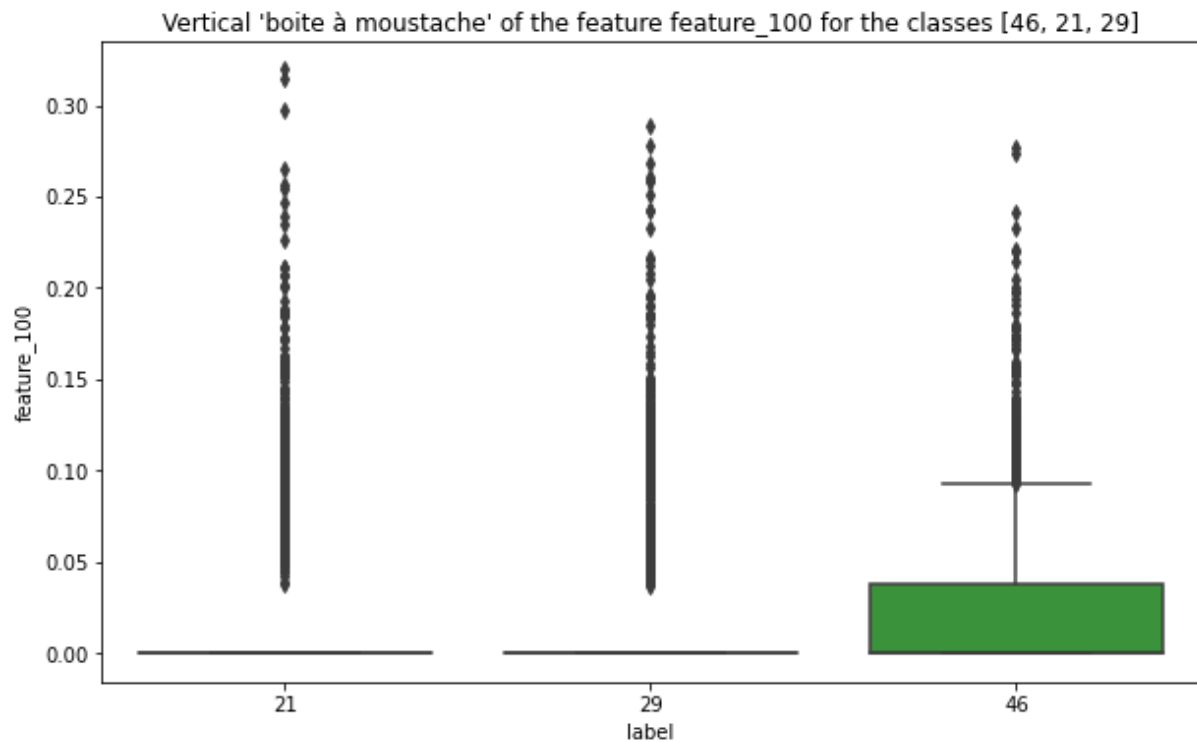


Figure 6: vertical 'boîte à moustache' of the feature 100 for the class [46,21,29]

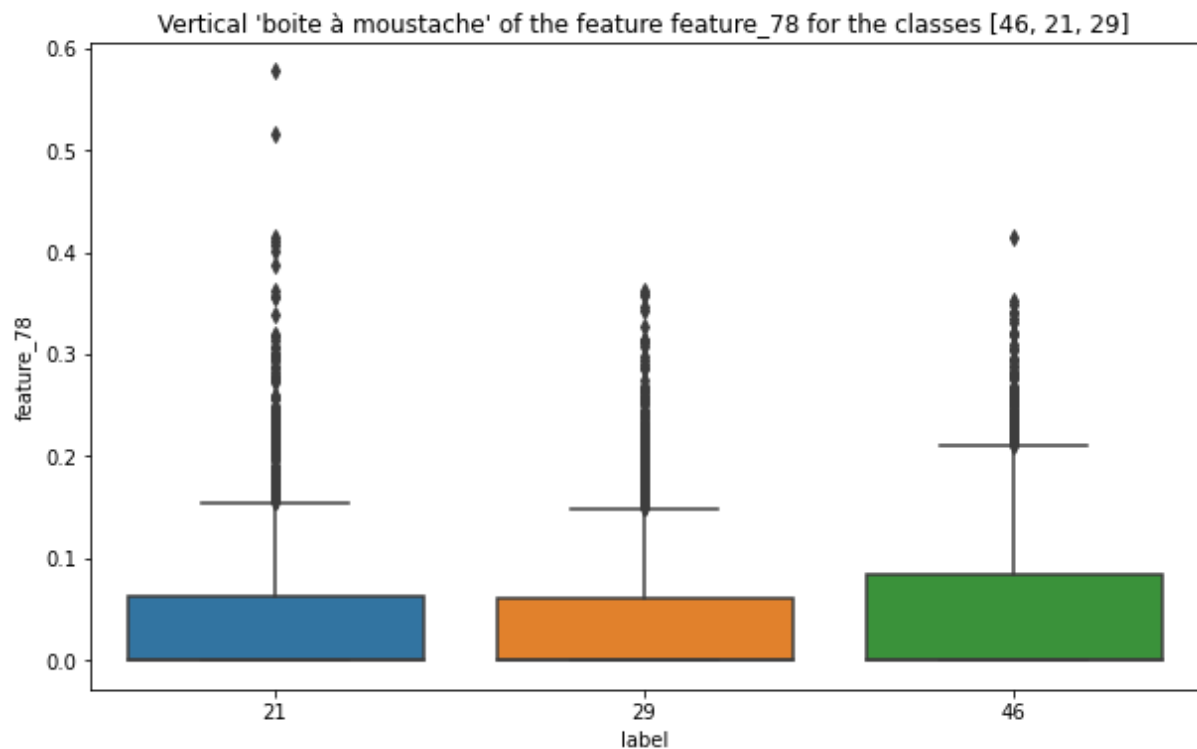


Figure 7 : vertical 'boîte à moustache' of the feature 78 for the class [46,21,29]



**architecture with five fully connected layers** was retained because it offered a good compromise between representational capacity, training time, and overfitting.

The influence of hidden layer size was examined in a similar way by varying the number of neurons per layer, for example comparing 128, 256, and 512 units. Larger layers allowed the model to fit the training data more closely and produced slightly higher training accuracy, but beyond a certain size the validation accuracy either plateaued or decreased, again signalling overfitting. Smaller layers, on the other hand, trained faster but sometimes failed to capture enough structure in the high dimensional feature space. Based on these experiments, an intermediate width was selected as the default architecture, since it provided stable validation performance while keeping the number of parameters and the risk of overfitting under control.

## VI. Conclusion

Through this project, the behaviour of several modelling approaches on a challenging fifty class classification task has been examined. The decision tree baseline that was implemented from scratch obtained a very low accuracy on the training set and proved unsuitable for the high dimensional feature space and the large number of classes. In contrast, a carefully designed multi-layer perceptron, trained with the AdamW optimizer and regularized through batch normalization, dropout and weight decay, achieved an accuracy of about 61% on the validation folds and on the Kaggle public leaderboard. The close agreement between validation and leaderboard scores indicates that the final model generalizes reasonably well to unseen data within the constraints of the competition.

The analysis of precision and recall by class, together with the confusion matrix and box plot visualizations, showed that certain classes are consistently well recognized, whereas others present systematic confusion patterns. Several classes share overlapping feature distributions, which leads to reduced recall or precision for those categories. The gap between training and validation performance confirms that some overfitting remains, even with the regularization techniques that were employed. Future work could include the exploration of alternative architectures for tabular data, such as attention-based models, a more targeted treatment of difficult classes, and refined regularization schedules. Nevertheless, the final model satisfies the project requirements, provides a solid baseline for this dataset, and illustrates the practical trade-offs involved when designing neural network models for high dimensional supervised learning tasks.



## VII. Appendix

### Precision and recall on the last version of the code

on the validations set:

K-FOLD ACCURACIES: [0.605276839095399, 0.6113686582037173, 0.6066461591785451, 0.6099389107924267, 0.610675447337637]

BEST FOLD ACCURACY: 0.6113686582037173

Train Set Accuracy (Best Model): 0.9195

### Train Set Classification Report:

|          | precision | recall | f1-score | support |
|----------|-----------|--------|----------|---------|
| Class 0  | 0.94      | 0.91   | 0.92     | 2309    |
| Class 1  | 0.91      | 0.90   | 0.91     | 2306    |
| Class 2  | 0.93      | 0.93   | 0.93     | 2307    |
| Class 3  | 0.94      | 0.92   | 0.93     | 2309    |
| Class 4  | 0.90      | 0.92   | 0.91     | 2309    |
| Class 5  | 0.96      | 0.96   | 0.96     | 2309    |
| Class 6  | 0.90      | 0.90   | 0.90     | 2310    |
| Class 7  | 0.89      | 0.92   | 0.90     | 2307    |
| Class 8  | 0.92      | 0.91   | 0.92     | 2304    |
| Class 9  | 0.91      | 0.89   | 0.90     | 2309    |
| Class 10 | 0.90      | 0.91   | 0.90     | 2307    |
| Class 11 | 0.94      | 0.94   | 0.94     | 2307    |
| Class 12 | 0.94      | 0.94   | 0.94     | 2310    |
| Class 13 | 0.96      | 0.96   | 0.96     | 2308    |
| Class 14 | 0.94      | 0.91   | 0.92     | 2307    |
| Class 15 | 0.91      | 0.92   | 0.92     | 2303    |
| Class 16 | 0.90      | 0.92   | 0.91     | 2305    |
| Class 17 | 0.94      | 0.92   | 0.93     | 2309    |
| Class 18 | 0.94      | 0.93   | 0.94     | 2309    |
| Class 19 | 0.92      | 0.91   | 0.92     | 2309    |
| Class 20 | 0.92      | 0.92   | 0.92     | 2310    |
| Class 21 | 0.92      | 0.87   | 0.90     | 2307    |
| Class 22 | 0.87      | 0.88   | 0.88     | 2310    |

|          |      |      |      |      |
|----------|------|------|------|------|
| Class 23 | 0.89 | 0.91 | 0.90 | 2309 |
| Class 24 | 0.90 | 0.91 | 0.90 | 2309 |
| Class 25 | 0.93 | 0.94 | 0.94 | 2304 |
| Class 26 | 0.90 | 0.88 | 0.89 | 2308 |
| Class 27 | 0.91 | 0.91 | 0.91 | 2310 |
| Class 28 | 0.94 | 0.94 | 0.94 | 2309 |
| Class 29 | 0.86 | 0.88 | 0.87 | 2310 |
| Class 30 | 0.90 | 0.91 | 0.91 | 2310 |
| Class 31 | 0.90 | 0.91 | 0.91 | 2308 |
| Class 32 | 0.95 | 0.95 | 0.95 | 2310 |
| Class 33 | 0.90 | 0.93 | 0.92 | 2310 |
| Class 34 | 0.90 | 0.92 | 0.91 | 2309 |
| Class 35 | 0.92 | 0.92 | 0.92 | 2309 |
| Class 36 | 0.91 | 0.89 | 0.90 | 2307 |
| Class 37 | 0.91 | 0.92 | 0.92 | 2305 |
| Class 38 | 0.93 | 0.90 | 0.91 | 2309 |
| Class 39 | 0.92 | 0.93 | 0.93 | 2307 |
| Class 40 | 0.90 | 0.90 | 0.90 | 2310 |
| Class 41 | 0.89 | 0.91 | 0.90 | 2310 |
| Class 42 | 0.92 | 0.92 | 0.92 | 2308 |
| Class 43 | 0.92 | 0.92 | 0.92 | 2308 |
| Class 44 | 0.91 | 0.92 | 0.92 | 2306 |
| Class 45 | 0.96 | 0.93 | 0.95 | 2310 |
| Class 46 | 0.97 | 0.97 | 0.97 | 2307 |
| Class 47 | 0.93 | 0.94 | 0.93 | 2305 |
| Class 48 | 0.96 | 0.96 | 0.96 | 2310 |
| Class 49 | 0.96 | 0.94 | 0.95 | 2309 |

|              |      |      |      |        |
|--------------|------|------|------|--------|
| accuracy     |      |      | 0.92 | 115406 |
| macro avg    | 0.92 | 0.92 | 0.92 | 115406 |
| weighted avg | 0.92 | 0.92 | 0.92 | 115406 |

Validation Set Accuracy (Best Model): 0.6114

## Validation Set Classification Report:

|          | precision | recall | f1-score | support |
|----------|-----------|--------|----------|---------|
| Class 0  | 0.65      | 0.59   | 0.62     | 453     |
| Class 1  | 0.59      | 0.55   | 0.57     | 491     |
| Class 2  | 0.65      | 0.65   | 0.65     | 438     |
| Class 3  | 0.68      | 0.63   | 0.65     | 462     |
| Class 4  | 0.59      | 0.64   | 0.62     | 491     |
| Class 5  | 0.79      | 0.77   | 0.78     | 425     |
| Class 6  | 0.49      | 0.48   | 0.48     | 443     |
| Class 7  | 0.54      | 0.62   | 0.57     | 471     |
| Class 8  | 0.59      | 0.55   | 0.57     | 450     |
| Class 9  | 0.56      | 0.51   | 0.53     | 495     |
| Class 10 | 0.54      | 0.55   | 0.54     | 471     |
| Class 11 | 0.72      | 0.71   | 0.72     | 468     |
| Class 12 | 0.73      | 0.73   | 0.73     | 471     |
| Class 13 | 0.81      | 0.80   | 0.81     | 454     |
| Class 14 | 0.68      | 0.59   | 0.63     | 478     |
| Class 15 | 0.58      | 0.62   | 0.60     | 434     |
| Class 16 | 0.55      | 0.61   | 0.58     | 445     |
| Class 17 | 0.65      | 0.61   | 0.63     | 445     |
| Class 18 | 0.70      | 0.66   | 0.68     | 449     |
| Class 19 | 0.61      | 0.59   | 0.60     | 478     |
| Class 20 | 0.62      | 0.62   | 0.62     | 476     |
| Class 21 | 0.55      | 0.42   | 0.47     | 470     |
| Class 22 | 0.44      | 0.46   | 0.45     | 493     |
| Class 23 | 0.52      | 0.58   | 0.55     | 477     |
| Class 24 | 0.51      | 0.54   | 0.52     | 447     |
| Class 25 | 0.67      | 0.70   | 0.69     | 443     |
| Class 26 | 0.50      | 0.45   | 0.48     | 491     |
| Class 27 | 0.53      | 0.54   | 0.53     | 436     |
| Class 28 | 0.73      | 0.74   | 0.73     | 473     |
| Class 29 | 0.41      | 0.44   | 0.43     | 479     |
| Class 30 | 0.52      | 0.55   | 0.53     | 440     |
| Class 31 | 0.52      | 0.57   | 0.54     | 435     |
| Class 32 | 0.75      | 0.75   | 0.75     | 447     |
| Class 33 | 0.60      | 0.68   | 0.64     | 495     |

|              |      |      |      |       |
|--------------|------|------|------|-------|
| Class 34     | 0.54 | 0.61 | 0.57 | 454   |
| Class 35     | 0.60 | 0.60 | 0.60 | 471   |
| Class 36     | 0.56 | 0.51 | 0.53 | 495   |
| Class 37     | 0.53 | 0.59 | 0.56 | 400   |
| Class 38     | 0.59 | 0.50 | 0.54 | 459   |
| Class 39     | 0.60 | 0.65 | 0.62 | 424   |
| Class 40     | 0.50 | 0.50 | 0.50 | 453   |
| Class 41     | 0.53 | 0.57 | 0.55 | 484   |
| Class 42     | 0.58 | 0.58 | 0.58 | 432   |
| Class 43     | 0.63 | 0.63 | 0.63 | 482   |
| Class 44     | 0.62 | 0.62 | 0.62 | 486   |
| Class 45     | 0.78 | 0.69 | 0.73 | 464   |
| Class 46     | 0.85 | 0.84 | 0.84 | 462   |
| Class 47     | 0.67 | 0.73 | 0.70 | 461   |
| Class 48     | 0.81 | 0.79 | 0.80 | 473   |
| Class 49     | 0.78 | 0.72 | 0.75 | 467   |
|              |      |      |      |       |
| accuracy     |      | 0.61 |      | 23081 |
| macro avg    | 0.61 | 0.61 | 0.61 | 23081 |
| weighted avg | 0.61 | 0.61 | 0.61 | 23081 |